

COLECCIONES DE DATOS

Cuando utilizamos un array estamos obligados a solicitar “espacio” para un número determinado de elementos y todos del mismo tipo. Quizás en tiempo de ejecución no necesitemos toda la memoria solicitada o por el contrario necesitamos más.

JAVA al igual que el resto de lenguajes orientados a objetos, dispone de clases que nos permiten almacenar igual que un array pero se incrementan automáticamente cuándo ocupamos todo el espacio inicial. Dichas clases se llaman colecciones. Se encuentran en el paquete **util**.

COLECCIONES

- **Collection<E>** Es un grupo de elementos individuales, frecuentemente con alguna regla aplicada a ellos.
- **List<E>**: Elementos en una secuencia particular que mantienen un orden y permite duplicados. La lista puede ser recorrida en ambas direcciones. Tipos:
 - **ArrayList<E>**: Su ventaja es que el acceso a un elemento en particular es ínfimo. Su desventaja es que para eliminar un elemento, se ha de mover toda la lista para eliminar ese “hueco”.
 - **LinkedList<E>**: En esta, los elementos están conectados con el anterior y posterior. La ventaja es que es fácil mover/eliminar elementos de la lista, simplemente moviendo/eliminando sus referencias hacia otros elementos. La desventaja es que para usar el elemento N de la lista, debemos realizar N movimientos a través de la lista.
- **Map<K,V> y HashMap<K,V>**. Un grupo de pares objetos clave-valor, que no permite duplicados en sus claves. Es quizás el más sencillo, y no utiliza la interfaz Collection. Los principales métodos son: **put()**, **get()**, **remove()**.
- **Set<E> y HashSet<E>** No puede haber duplicados, Cada elemento debe ser único, por lo que si existe uno duplicado, no se agrega, Por regla general, cuando se redefine **equals()**, se debe redefinir **hashCode()**. Es necesario redefinir **hashCode()** cuando la clase definida será colocada en un **HashSet**. Los métodos **add(o)** y **addAll(o)** devuelven false si ya estaba en el conjunto.
- **Queue<E>**: Colección ordenada con extracción por el principio el inserción por el principio (LIFO- Last Input, First Output) o por el final (FIFO - First Input, First Output). Se permiten elementos duplicado. No da excepciones cuando la cola está vacía/llena, hay métodos para interrogar, que devuelven null.

ARRAYLIST

Un **ArrayList** es un objeto que sirve para almacenar otros objetos. Es parecido a un array pero se diferencia en:

- Un **ArrayList** crece y decrece automáticamente dependiendo de los elementos que tenga almacenados. No es necesario darle el n° de elementos al construirlo. En realidad internamente lleva un array para almacenar los elementos. Cuando se supera la capacidad inicial se crea otro array mayor y se copian todos los elementos, todo esto es transparente para el usuario, pero puede ser costoso si se hace frecuentemente.
- Un array forma parte del lenguaje Java, un ArrayList es una clase del paquete util.
- Un array puede contener tipos básicos u objetos, un ArrayList sólo contiene objetos, En los ejemplos usaremos Object para representar a cualquier tipo de clase.
- En otras palabras tarda menos en buscar un elemento

CREAR UN ARRAYLIST

- **ArrayList <Object> nombre = new ArrayList<Object>();**

Ej, ArrayList de Integer (Objeto de enteros)

ArrayList <Integer> numero = new ArrayList <Integer>();

MÉTODOS DE ARRAYLIST/LINKEDLIST

FUNCIÓN	SINTAXIS	DESCRIPCIÓN	EJEMPLOS
TAMAÑO	int size();	Retorna el número de elementos almacenados en el ArrayList. Inicialmente 0. (Devuelve int) Método no estático	Sysout(numeros.size()); -> 0
AÑADIR UN ELEMENTO	boolean add(Object obj) Si hemos parametrizado Si no no funcionara.	Añade un elemento al final de la lista (Devuelve true si pudo y false si no pudo) (Solo false si te quedas sin memoria) (Ahorra la búsqueda)	numeros.add(5);
	void add(int index, Object obj)	Añade un elemento en la posición index (entre 0 y size()) y desplaza todos los elementos a la derecha No	numeros.add(2,5))

		devuelve nada (Requiere buscar la posición)	
OBTENER UN ELEMENTO	E.get(int index)	Devuelve el objeto almacenado en la posición index del ArrayList, el índice debe estar comprendido entre 0 y size()-1. No saca al elemento del arraylist. (Devuelve un objeto)	Personas.get(2);
MODIFICAR UN ELEMENTO	E.set (int index, Object obj)	Mueve el valor de obj a la posición index del arraylist. La posición debe estar comprendida entre 0 y size()-1. Devuelve el objeto. Si no existe excepción.	numeros.set(1,n);
BORRAR UN ELEMENTO	E.remove (int index)	Borra un elemento de la posición index (de 0 a size()-1) y mueve todos los elementos a la izquierda. Devuelve el elemento borrado.	numeros.remove(2)
	boolean remove(Object obj)	Borra el objeto obj en el ArrayList. Si lo encuentra retorna true, si no lo encuentra retorna false. Utiliza el método equals para comparar los objetos. (Generar el método equals en la clase). (Ahorra la búsqueda)	numeros.remove(n 2);
	e.clear()	Borra todos los	numeros.clear()

		elementos del ArrayList (No devuelve nada)	
BUSCAR UN ELEMENTO (NOTA): Al usar tipos primitivos no es necesario usar equals (Ya tienen definidos sus equals) cuando se usa las versiones clase de estas variables.	boolean contains (Object obj)	Busca un objeto o variable en el ArrayList y devuelve true si lo encuentra en el array y false en caso contrario. Usa el método equals. (Generar en clase dicho método) (Se ahorra la búsqueda por for)	if(numeros.contains(n)) { }
	int indexOf (Object obj)	Usando el método equals nos dice si ya existe un elemento en un ArrayList. Retorna -1 si no lo encuentra y si lo encuentra el índice donde lo ha encontrado	int pos = indexOf(n)
VERIFICAR SI EL ARRAYLIST ESTA VACIO	boolean object.isEmpty()	Corrobora si la lista esta vacia o no false si no true si si	

NOTAS DE LA TABLA

- SIEMPRE SE USA EL NOMBRE DE LA COLECCIÓN PARA EL USO DE LOS MÉTODOS
- EN TODOS LOS CASOS, SI SE ACCEDER A UNA POSICIÓN FUERA DEL ARRAYLIST, SALTA LA EXCEPCIÓN INDEXOUTOFBOUNDSEXCEPTION.
- PARA CLONAR UNA LISTA HACEMOS
ArrayList<Alumno> copiaLista = new ArrayList<>(listaAlumnos);

EJEMPLO USO DE ARRAYLIST

```
public class Main {  
    public static void main(String[] args) {  
        ArrayList<String> cars = new ArrayList<String>();  
        cars.add("Volvo");  
        cars.add("BMW");  
        cars.add("Ford");  
        cars.add("Mazda");  
        for (int i = 0; i < cars.size(); i++) {  
            System.out.println(cars.get(i));  
        }  
    }  
}
```

LINKEDLIST

Funcionan igual que los **ArrayList**, y tienen los mismos métodos

Se diferencia en como se almacenan internamente los datos. En este caso se almacenan en una lista enlazada, cada elemento tiene referencia al anterior y posterior.

Por este motivo, en un **ArrayList** el acceso a un elemento es muy rápido, mientras que el borrado y la inserción son costosos. Justo al revés en un **LinkedList**, dónde insertar y borrar es sencillo pero acceder a un elemento es costoso porque tiene que recorrerse todos los anteriores.

POR LO TANTO LO MEJOR ES USAR ARRAYLIST PARA ALMACENAR Y ACCEDER A DATOS Y USAR LINKEDLIST CUANDO VAMOS A HACER MUCHAS ALTAS Y MODIFICACIONES

Que estructura de datos es mejor usar:

Acceso rápido:

- **ArrayList:** Excelente para acceder a elementos por índice.
- **LinkedList:** Acceso secuencial

Inserción/ Eliminación

- **ArrayList:** Mejor al final; más costoso en medio/inicio.
- **LinkedList:** Eficiente para inserciones y borrados en cualquier posición (si ya tienes la referencia), pero la búsqueda por índice es lenta).

La clase **LinkedList** aparte de tener los mismos métodos que el **ArrayList** posee algunos más para operaciones más eficientes:

- **addFirst(Object obj):** Añade un elemento al principio
- **addLast(Object obj):** Añade un elemento al final.

- **removeFirst():** Borra el primer elemento
- **removeLast():** Borra el ultimo elemento.

Ejemplo uso de LinkedList:

```
public class Main {
    public static void main(String[] args) {
        LinkedList<String> cars = new LinkedList<String>();
        cars.add("Volvo");
        cars.add("BMW");
        cars.add("Ford");
        cars.add("Mazda");
        for (int i = 0; i < cars.size(); i++) {
            System.out.println(cars.get(i));
        }
    }
}
```

HASHMAP (NO ES UN ARRAY NO USA POSICIONES)

Es como una tabla que almacena los datos en parejas: clave,valor.

Se usa un objeto para la clave y otro para el valor. No puede haber 2 elementos con la misma clave. PUEDEN SER OBJETOS DEFINIDOS POR JAVA O PUEDEN SER OBJETOS DEFINIDOS POR MI

Ejemplo de HashMap, que almacena parejas nombre, edad. La clave es el nombre:

HashMap<String, Integer> people = new HashMap<String, Integer>();

Usa equals() necesario codificarlo si lo usas con clases creadas por nosotros pero no necesario para otros objetos.

NO USA POSICIONES

Almacena los valores según el **hashCode** de la clave.

Brilla en los siguientes escenarios:

- **Busqueda por Etiqueta (ID o Nombre)**
- **Contadores y Estadísticas**
- **Diccionarios o traducciones**

MÉTODOS MÁS IMPORTANTES:

- **put(clave,valor):** Introduce una clave con su valor asociado. Si existe esa clave sobrescribe el valor.
- **get(clave):** Accede al valor asociado a esa clave, si no existe devuelve null.
- **containsKey(clave):** Devuelve true o false dependiendo si existe un dato con esa clave o no.
- **remove(clave):** Borra el dato con dicha clave.

**NO SE PUEDE RECORRER CON UN FOR NORMAL Y NO ES RECOMENDABLE
GENERALMENTE LO IDEAL ES SOLO USAR UN FOR:EACH PARA LA CLAVE Y EL
VALOR.**

EJEMPLO HASHMAP:

```
public class Main {  
    public static void main(String[] args) {  
        HashMap<String, Integer> people = new HashMap<String, Integer>()  
  
        people.put("John", 32);  
        people.put("Steve", 30);  
        people.put("Angie", 33);  
  
        // Nos recorreremos las claves con el for, y por cada clave accedemos al valor con get.  
  
        for (String i : people.keySet()) {  
            System.out.println("key: " + i + " value: " + people.get(i));  
        }  
    }  
}
```

HASHSET (NO ES UN ARRAY NO USA POSICIONES)

Es una colección de datos dónde cada dato es único.

Si estoy guardando objetos que no son String, ni Integer ni Double, es imprescindible añadir al método hashCode e equals para saber qué campos o combinación de campos tiene que ser único.

Ejemplo:

```
HashSet<String> cars = new HashSet<String>();
```

MÉTODOS MÁS IMPORTANTES

- **add(obj):** Añade un objeto si no existe. Devuelve false si ya existe y no lo puede insertar, true en otro caso.
- **contains(obj):** Indica si el objeto está en el conjunto. Otros tipos de datos son transformados automáticamente.
- **size():** Número de elementos
- **remove(obj):** Borra un objeto.

Se recorre con for_each

EJEMPLO HASHSET:

```
HashSet<Integer> numbers = new HashSet<Integer>();

// Add values to the set
numbers.add(4);
numbers.add(7);
numbers.add(8);
// Recorrer el set
System.out.println("Elementos del conjunto:");
for (Integer i:numbers)
    System.out.println(i);
```

FOR-EACH:

El for-each es básicamente la versión “perezosa” y elegante del **for** tradicional. Se inventó para que no tengamos que estar peleándonos con los índices (i), el size() y los límites del array.

ESTRUCTURA (SINTAXIS)

Imaginemos que tenemos un **ArrayList<String> nombres**. el for-each se escribe así

```
for (String aux : nombres) {
    System.out.println(aux);
}
```


SE LEE ASÍ:

Por cada String (al que llamaremos aux) que encuentres dentro de nombres, ejecuta el código entre llaves).

QUE SIGNIFICA CADA PARTE?

- **String:** Es el tipo de dato de los elementos que hay dentro de la colección. Si la lista es de Integer, aquí pones Integer.
- **aux:** Es una variable temporal (Se puede llamar como sea) En cada vuelta del bucle, esta variable “copia” el valor del elemento actual.
- **:** Significa dentro de o de la colección
- **nombres:** Es el nombre de tu **ArrayList**, **HashSet** o **Array** que quieres recorrer.

DIFERENCIA CON EL FOR NORMAL

FOR	FOR-EACH
<code>for (int i = 0; i < lista.size(); i++)</code>	<code>for (String s : lista)</code>
Tienes que usar <code>lista.get(i)</code> para ver el dato	El dato ya viene “masticado” en la variables s. (el dato puede ser cualquier tipo) NO PUEDES AÑADIR O ELIMINAR DATOS
Puedes saltarte elementos (ej. de 2 en 2)	Siempre recorre todos, uno por uno
Sirve para <code>ArrayList</code> y <code>Arrays</code>	Sirve para <code>ArrayList</code> , <code>Arrays</code> , <code>Hashset</code> y <code>LinkedList HashMap</code>
Hay “mutiples formas” de detenerlo aunque son algo tediosas	La mejor forma de detenerlo es con un <code>break</code>

ITERATOR:

Es un objeto que se puede usar para recorrer colecciones, como **ArrayList** y **HashSet** se construye desde cualquier colección. tiene usos por lo general con `ArrayList` y `LinkedList` las otras pueden borrar sin necesidad de bucles.

Ejemplo:

```

ArrayList<String> cars = new ArrayList<String>();
cars.add("Volvo");
cars.add("BMW");
cars.add("Ford");
cars.add("Mazda");

// Get the iterator
Iterator<String> it = cars.iterator();

// Loop through a collection
while(it.hasNext()) {
    System.out.println(it.next());
}

```

MÉTODOS MÁS IMPORTANTES (SE USA EL NOMBRE DEL ITERATOR) (SOLO PARA BORRAR Y MOSTRAR)

- **hasNext()** : Devuelve un boolean que indica si hay más elementos
- **next()**: Devuelve el elemento siguiente.
- **remove()**: Borra el elemento (Mejor que usar for o for-each)

CLASES Y MÉTODOS GENÉRICOS

En Java, los **genéricos**: permiten definir clases y métodos de forma **parametrizada**, de manera que pueden trabajar con diferentes tipos de datos sin necesidad de definir una versión específica para cada tipo. Esto aumenta la reutilización del código, mejora la seguridad en tiempo de compilación y reduce la necesidad de hacer conversiones de tipo (casting).

EJEMPLOS DE CLASE GENÉRICA:

CLASE:

Ejemplo de clase genérica:

```

// Definimos una clase genérica Caja<T>
class Caja<T> {
    private T contenido;

    public void setContenido(T contenido) {
        this.contenido = contenido;
    }
}

```

```

        public T getContenido() {
            return contenido;
        }
    }
}

```

MAIN:

```

public class Main {
    public static void main(String[] args) {
        // Creamos una Caja de tipo String
        Caja<String> cajaString = new Caja<>();
        cajaString.setContenido("Hola");
        System.out.println("Contenido: " + cajaString.getContenido());

        // Creamos una Caja de tipo Integer
        Caja<Integer> cajaInt = new Caja<>();
        cajaInt.setContenido(42);
        System.out.println("Contenido: " + cajaInt.getContenido());
    }
}

```

T es un parámetro de tipo genérico

Caja<T> es una clase que puede almacenar cualquier tipo de dato

Al instanciar la clase, se define el tipo real (Caja <String> o Caja <Integer>).

Ejemplo de método genérico

El método **imprimirArray(T[] array)** puede recibir cualquier tipo de array.

CLASE

```

class Utilidades {
    // Método genérico que imprime cualquier tipo de array
    public static <T> void imprimirArray(T[] array) {
        for (T elemento : array) {
            System.out.print(elemento + " ");
        }
        System.out.println();
    }
}

```

MAIN

```

public class Main {
    public static void main(String[] args) {
        String[] nombres = {"Ana", "Luis", "Carlos"};
        Integer[] numeros = {1, 2, 3, 4, 5};

        Utilidades.imprimirArray(nombres);
        Utilidades.imprimirArray(numeros);
    }
}

```

OPERACIONES AGREGADAS EN COLECCIONES

(SE VEN MEJOR CON HERENCIA)

Las **operaciones agregadas** en Java son métodos que permiten realizar cálculos y transformaciones sobre colecciones de datos de manera **concisa y eficiente**. Estas operaciones se introdujeron en **Java 8** con la API de **Streams**, lo que permite escribir código más declarativo y reducir la cantidad de bucles explícitos.

Las operaciones agregadas más comunes en colecciones son:

forEach() -> Aplica una acción a cada elemento de la colección

map() -> Transforma los elementos de una colección en otros valores

filter() -> Filtra elementos que cumplen una condición

collect() -> Convierte un Stream en otra estructura de datos (como una lista)

reduce() Cuenta los elementos en una colección

min()/max() Encuentra el mínimo o máximo en una colección.

Ejemplo agregadas con LinkedList

```
import java.util.LinkedList;
import java.util.List;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        List<Integer> numeros = new LinkedList<>();
        numeros.add(5);
        numeros.add(10);
        numeros.add(15);
        numeros.add(20);
        numeros.add(25);

        // Sumar todos los números
        int suma = numeros.stream().reduce(0, Integer::sum);
        System.out.println("Suma total: " + suma); // Suma total: 75

        // Obtener solo los números pares
        List<Integer> pares = numeros.stream()
            .filter(n -> n % 2 == 0)
            .collect(Collectors.toList());

        System.out.println(pares); // [10, 20]
    }
}
```

Ejemplo agregadas con ArrayList

```

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        List<String> nombres = new ArrayList<>();
        nombres.add("Ana");
        nombres.add("Luis");
        nombres.add("Carlos");
        nombres.add("Lucía");
        nombres.add("Laura");

        // Filtrar nombres que empiezan con "L"
        List<String> nombresConL = nombres.stream()
            .filter(nombre -> nombre.startsWith("L"))
            .collect(Collectors.toList());

        System.out.println(nombresConL); // [Luis, Lucía, Laura]

        // Convertir todos los nombres a mayúsculas
        List<String> nombresMayus = nombres.stream()
            .map(String::toUpperCase)
            .collect(Collectors.toList());

        System.out.println(nombresMayus); // [ANA, LUIS, CARLOS, LUCÍA, LAURA]
    }
}

```

Ejemplo agregadas con HashMap

```

import java.util.HashMap;
import java.util.Map;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        Map<String, Integer> edades = new HashMap<>();
        edades.put("Ana", 25);
        edades.put("Luis", 30);
        edades.put("Carlos", 28);
        edades.put("Lucía", 22);

        // Obtener nombres de personas mayores de 25 años
        Map<String, Integer> mayoresDe25 = edades.entrySet().stream()
            .filter(entry -> entry.getValue() > 25)

```

Ejemplo agregadas con HashSet

```

import java.util.HashSet;
import java.util.Set;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        Set<Integer> numeros = new HashSet<>();
        numeros.add(10);
        numeros.add(15);
        numeros.add(20);
        numeros.add(25);
        numeros.add(30);

        // Obtener solo los números mayores de 15
        Set<Integer> mayoresDe15 = numeros.stream()
            .filter(n -> n > 15)
            .collect(Collectors.toSet());

        System.out.println(mayoresDe15); // [20, 25, 30]

        // Obtener la suma total
        int sumaTotal = numeros.stream().reduce(0, Integer::sum);
        System.out.println("Suma total: " + sumaTotal); // Suma total: 100
    }
}

```

HASHCODE

El hashCode de un objeto es un número entero que se calcula mediante un algoritmo.

Sirve para facilitar las búsquedas.

Por ejemplo el hashCode de un String se calcula mediante este algoritmo

```

public int hashCode() {
    int h = 0;
    for (int i = 0; i < this.length(); i++) {
        h = 31 * h + this.charAt(i);
    }
    return h;
}

```

Dos objetos distintos pueden tener el mismo hashCode

Por ejemplo “FB” y “Ea” ó “Aa” y “BB” tienen el mismo hashCode

Para buscar una estructura hash primero se busca por hashCode y entre el total de objetos que tienen el mismo hashCode se busca por equals

Cuando mi estructura hash tiene como clave una clase propia, tengo que definirme el hashCode para mi clase y el equals, en otro caso no funciona.

Si la clave es un String o un Integer no hace falta porque estas clases ya tienen definido estos métodos.

RESUMEN DE USOS

SI NECESITO	USAMOS	POR QUÉ?
Acceso rápido por índice (ej: “dame el 5º)	ArrayList	Es como un array: vas directo a la posición
Hacer muchas altas/bajas (sobre todo al inicio)	LinkedList	Solo cambia “hilos”, no mueve todo el bloque no recomendable en elementos mas (Al centro)
Buscar por su ID o Nombre (Clave-Valor)	HashMap	El hashCode te lleva directo al dato sin buscar
Evitar duplicados (Saco de elementos únicos)	HashSet	Usa el hashCode para asegurar que nadie se repita
Estructura LIFO o FIFO (Pilas o Colas)	Queue	Está optimizado para entrar y salir por los extremos.

REALMENTE SE PUEDEN INTERCAMBIAR ENTRE ELLOS?

A.INTERCAMBIO POR FAMILIA (LISTAS)

Como el ArrayList y el LinkedList son hermanos (ambos implementan la interfaz List) puedes intercambiarlos fácilmente porque tienen los mismos métodos (add,get,remove)

TRUCO DE CONVERSIÓN

Java

```
// Pasar de LinkedList a ArrayList  
ArrayList<String> miLista = new ArrayList<>(miLinkedList);
```

B.INTERCAMBIO ENTRE ESPECIES DIFERENTES (MAPA A LISTA)

No se puede convertir un **HashMap** en un **ArrayList** directamente (porque uno tiene parejas Clave-Valor y el otro no). **Pero puedes volcar los datos**

- Puedes sacar todas las **Claves** y meterlas en un **ArrayList**
- Puedes sacar todos los **Valores** y meterlos en un **ArrayList**

Se puede pasar de ArrayList a HashSet y viceversa.

ANOTACIONES:

- Se puede recorrer con un for regular el ArrayList y el LinkedList pero no un **HashMap** o un **HashSet**. un for-each sirve para todos.
- De poder se pueden meter colecciones dentro de otras pero para lo que necesito hasta ahora es “complicarse la vida”.